



Computational Logic in the Era of AI

Propositional and Predicate Logic in λC

Sergey Mechtaev

mechtaev@pku.edu.cn

Term 1 • 26–27

This lecture

↓ you are here



1

Absurdity

$\perp := \Pi \alpha : * . \alpha$,
 $\neg A := A \rightarrow \perp$

False, Not

- ▶ A false proposition is an empty type, and from an inhabitant of the empty type anything follows.

2

Connectives

$\wedge, \vee, \Leftrightarrow$ as
 Π -types over all
propositions

And, Or, Iff

- ▶ The rules of a connective are theorems derived from its coding, not axioms.

3

Existence

$\exists := \Pi \alpha : * . ((\Pi x : S . P x \rightarrow \alpha) \rightarrow \alpha)$

Exists, $\langle w, h \rangle$

- ▶ A proof of $\exists$ carries a witness, and using it means raising a flag for that witness.

4

Classical

the excluded third
as an assumed
inhabitant i_{ET}

Classical.em

- ▶ Excluded middle is not derivable; it is assumed, and every proof that uses it says so.

5

Goals

the goal first:
 $? : A$, then the
lines above it

by, intro, apply,
exact

- ▶ A proof is found from its goal upward, and the machine keeps the flags and the holes.

History: the rules of logic, and where the extra one enters



L. E. J. Brouwer

1881–1966

A proof of $A \vee B$
must say which;
the excluded third
is not a law.

- 1908** Brouwer rejects the excluded third as a general law of reasoning.
- 1930** Heyting formalises the logic that results: intuitionistic, today called constructive.
- 1935** Gentzen: natural deduction, one introduction and one elimination rule per connective.
- 1969** Howard: the correspondence, extended to $\wedge$, $\vee$, $\supset$, $\forall$, $\exists$.
- 1972** Girard: system F, where the connectives, long known to be definable from $\forall$ and $\Rightarrow$, become λ -terms.
- 1985** Coquand and Huet: the same codings in λC . This lecture's chapter.



Gerhard Gentzen

1909–1945

Each connective
has a rule to prove
it and a rule to use
it.

Constructive logic is what the codings give for free; classical logic is one extra assumption, and this lecture shows where it enters. Photos: E. Menzler-Trott/MFO (CC BY-SA 2.0 de) and Wikimedia Commons.

N&G §7.4, §7.8

Minimal logic, and what is missing

Natural deduction for predicate logic has seven operations; λC has one type former, Π . The first two are Π -types with rules (*appl*) and (*abst*); the other five are Π -types too, and their rules are derived.

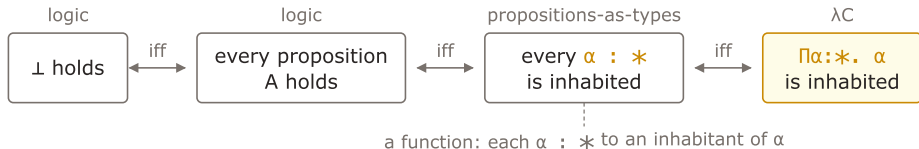
logic	coded in λC	its rules
$A \Rightarrow B$	$A \rightarrow B$	(<i>appl</i>), (<i>abst</i>)
$\forall_{x \in S} (P(x))$	$\Pi x:S. Px$	(<i>appl</i>), (<i>abst</i>)
$\perp$	?	derived
$\neg A$	?	derived
$A \wedge B$	?	derived
$A \vee B$	?	derived
$\exists_{x \in S} (P(x))$	?	derived

Derived means: the introduction and elimination rules of natural deduction become derivable rules of λC , and each derivation is a term.

N&G §7.1

Absurdity: from the property to the type

What characterises $\perp$ in logic: if $\perp$ holds, every proposition holds.



Definition. $\perp := \prod \alpha : *. \alpha$

A product over all propositions: a $\lambda 2$ type.

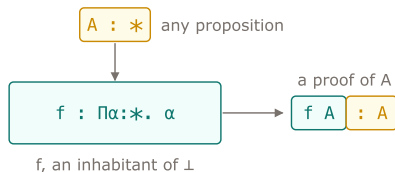
$$\frac{\vdash * : \square \quad \alpha : * \vdash \alpha : *}{\vdash \prod \alpha : *. \alpha : *} \text{ (form), pair } (\square, *)$$

The domain $*$ has sort $\square$; the body α has sort $*$. λP has no $(\square, *)$, so $\perp$ cannot be formed there.

N&G §7.1, Rem. 7.1.1

$\perp$ -elimination is one application

An inhabitant f of $\perp$ is a function on propositions: give it any $A : *$ and it returns a proof of A .



the same step, as an instance of (appl)

$$\frac{f : \prod \alpha : *. \alpha \quad A : *}{f A : \alpha[\alpha := A] = A} \text{ (appl)}$$

the bound α is replaced by the argument A ,
so the result has type A : a proof of A

$$\frac{\perp}{A} \text{ (}\perp\text{-elim)}$$

$$\begin{array}{ll} (a) & f : \prod \alpha : *. \alpha \\ (i) & f A : A \end{array} \quad \text{(appl) on (a), } A : *$$

The rule is not postulated. It is (appl) with a proposition as the argument.

N&G §7.1, Rem. 7.1.1

Negation

Definition

$\neg A := A \rightarrow \perp$, that is, $A \Rightarrow \perp$.

The rules of $\neg$ are the rules of $\Rightarrow$ with $\perp$ for B :

$$\frac{\text{Assume } A \cdots \perp}{\neg A} (\neg\text{-intro})$$
$$\frac{\neg A \quad A}{\perp} (\neg\text{-elim})$$

$$\frac{\text{Assume } A \cdots B}{A \Rightarrow B} (\Rightarrow\text{-intro})$$
$$\frac{A \Rightarrow B \quad A}{B} (\Rightarrow\text{-elim})$$

Instance, with $A : *$, $u : A \rightarrow \perp$, $v : A$: $uv : \perp$ by (*appl*). This is ($\neg$ -elim), and also ($\perp$ -intro).

Not an instance: $\neg S$ for a set $S : *$, although $S \rightarrow \perp$ is a well-formed type.

Next slide.

N&G §7.1, Rem. 7.1.2

False and Not, in Lean

```
-- §7.1:  $\perp$  :=  $\prod \alpha : *. \alpha$ , and  $\perp$ -elim is one (appl)
def Bot : Prop :=  $\forall \alpha : \text{Prop}, \alpha$ 
theorem bot_elim (f : Bot) (A : Prop) : A := f A

-- Lean: False has no constructor; Not is the same abbreviation
#print False      -- inductive False : Prop (no constructors)
#print Not        -- def Not : Prop → Prop := fun a => a → False
#check @False.elim -- {C : Sort u_1} → False → C
#check @absurd    -- {a : Prop} → {b : Sort u_1} → a →  $\neg a$  → b

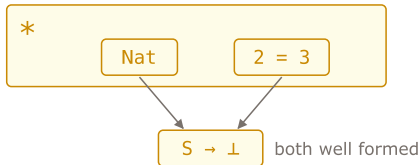
-- the two readings of "empty" agree, in both directions
example : False → Bot := fun h _ => h.elim
example : Bot → False := fun f => f False
```

Two readings of “empty”: False has no constructor, the coded $\perp$ has no closed inhabitant. The two terms at the bottom show they are the same proposition.

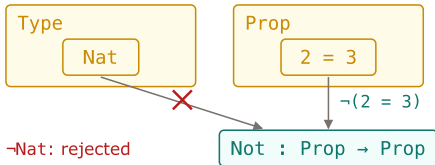
A set is not a statement

In λC , $\mathbb{N}$ and $2 = 3$ both have sort $*$, so $\neg$ applies to both. Lean splits $*$ into Type and Prop, and $\neg$ applies to Prop only.

λC : one sort



Lean: two sorts



```
#check (2 = 3) → False      -- 2 = 3 → False : Prop      ¬(2 = 3)
#check Nat → False         -- ∀ (a : Nat), False : Prop  "Nat is empty": a proposition too
example : (Nat → False) → False := fun h => h 0      -- and 0 refutes it
#check_failure ¬ Nat       -- Nat has type Type but is expected to have type Prop
```

$\mathbb{N} \rightarrow \perp$ is still a proposition, " $\mathbb{N}$ is empty"; what Lean refuses is to call it $\neg \mathbb{N}$.

Conjunction

Definition. $A \wedge B := \Pi C: * . (A \rightarrow B \rightarrow C) \rightarrow C$. Read Π as “for all” and $\rightarrow$ as “implies”:

for all C : if (A and B together) imply C , then C .

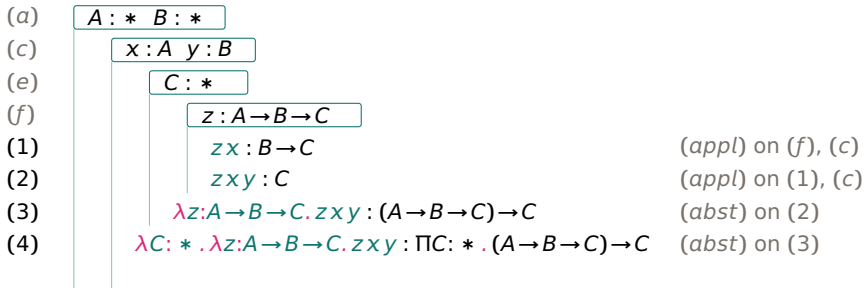
The condition “ A and B together” is redundant exactly when both hold. So the sentence says that both hold, and says it without a $\wedge$.

$$\begin{array}{c} \frac{A \quad B}{A \wedge B} \text{ (}\wedge\text{-intro)} \\ \frac{A \wedge B}{A} \text{ (}\wedge\text{-elim-left)} \end{array} \qquad \begin{array}{c} \frac{A \quad B}{\Pi C: * . (A \rightarrow B \rightarrow C) \rightarrow C} \text{ (}\wedge\text{-intro-sec)} \\ \frac{\Pi C: * . (A \rightarrow B \rightarrow C) \rightarrow C}{A} \text{ (}\wedge\text{-elim-left-sec)} \end{array}$$

The right-hand rules are the claim. They hold if, under the coding, a term of the conclusion’s type can be built from terms of the premisses’ types. N&G §7.2, I

$\wedge$ -intro-sec, derived

Find $?_1 : \Pi C : * . (A \rightarrow B \rightarrow C) \rightarrow C$ from $x : A$ and $y : B$. Bottom to top: the Π opens a flag for C , the $\rightarrow$ a flag for z .



And the eliminations, in one line each, with $u : A \wedge B$: $uA(\lambda v:A. \lambda w:B. v) : A$
and $uB(\lambda v:A. \lambda w:B. w) : B$.

N&G §7.2, Ex. 7.4

And, in Lean

The coding, and its three rules derived.

```
-- A ∧ B := ΠC:*. (A → B → C) → C
def And₂ (A B : Prop) : Prop := ∀ C : Prop, (A → B → C) → C

theorem And₂.intro {A B : Prop} (x : A) (y : B) : And₂ A B := fun _ z => z x y
theorem And₂.left {A B : Prop} (u : And₂ A B) : A := u A (fun x _ => x)
theorem And₂.right {A B : Prop} (u : And₂ A B) : B := u B (fun _ y => y)
```

Lean's And: an inductive type, the same three rules. The constructor is $\wedge$ -intro, the two fields are the eliminations.

```
#print And -- structure And (a b : Prop) : Prop
-- fields: And.left : a And.right : b
#check @And.intro -- ∀ {a b : Prop}, a → b → a ∧ b
example (A B : Prop) (h : A ∧ B) : B ∧ A := {h.right, h.left}
```

Every connective of this lecture is inductive in Lean; what that buys over the coding is a later lecture's topic.

Disjunction

Definition. $A \vee B := \Pi C : * . (A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow C.$

for all C : if A implies C and B implies C , then C .

Two cases, and in each C follows: that is what a proof from $A \vee B$ looks like. The coding is the elimination rule, spelled out.

$$\frac{A}{A \vee B} \text{ (}\vee\text{-intro-left)} \quad \frac{B}{A \vee B} \text{ (}\vee\text{-intro-right)} \quad \frac{A \vee B \quad A \Rightarrow C \quad B \Rightarrow C}{C} \text{ (}\vee\text{-elim)}$$

Instances, with $x : A$ and $u : A \vee B$, $v : A \rightarrow C$, $w : B \rightarrow C$:

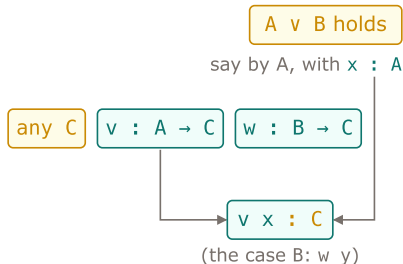
$\lambda C : * . \lambda v : A \rightarrow C . \lambda w : B \rightarrow C . v x : A \vee B$ and $u C v w : C$.

Not an instance: $u C v$ does not prove C . Its type is $(B \rightarrow C) \rightarrow C$: the case B is still owed, and the type says so. N&G §7.2, II

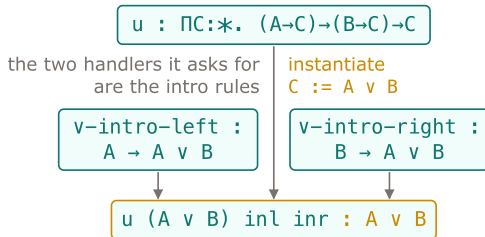
The coding says exactly $\vee$: both directions

Why this Π -type and no other: $A \vee B$ and its coding imply each other, and the argument each way is one step.

$A \vee B \Rightarrow \Pi C : *. (A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow C$



$\Pi C : *. (A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow C \Rightarrow A \vee B$



N&G §7.2, II

$\vee$ -elim-sec, derived

Find $? : C$ from $x : A \vee B$, $y : A \rightarrow C$, $z : B \rightarrow C$. Under the coding, x expects a proposition and two functions: three applications.

- | | | |
|-----|--|-----------------------------|
| (a) | $A : * \quad B : * \quad C : *$ | |
| (d) | $x : \prod D : *. (A \rightarrow D) \rightarrow (B \rightarrow D) \rightarrow D$ | |
| (e) | $y : A \rightarrow C \quad z : B \rightarrow C$ | |
| (1) | $x C : (A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow C$ | (<i>appl</i>) on (d), (a) |
| (2) | $x C y : (B \rightarrow C) \rightarrow C$ | (<i>appl</i>) on (1), (e) |
| (3) | $x C y z : C$ | (<i>appl</i>) on (2), (e) |

In (1) the argument C replaces the bound D in the type of x : the (*appl*) of $\lambda 2$.
The introductions are Exercise 7.7: open the coding's three flags, write $\vee x : C$,
close them with three (*abst*).

N&G §7.2, p. 143

Or, in Lean

```
-- A ∨ B :=  $\prod C : *$ . (A → C) → (B → C) → C
def Or₂ (A B : Prop) : Prop :=  $\forall C : \text{Prop}$ , (A → C) → (B → C) → C
theorem Or₂.inl {A B : Prop} (x : A) : Or₂ A B := fun _ f _ => f x
theorem Or₂.elim {A B C : Prop} (x : Or₂ A B) (y : A → C) (z : B → C) : C :=
  x C y z

#print Or
-- inductive Or : Prop → Prop → Prop
--   Or.inl : a → a ∨ b   Or.inr : b → a ∨ b
#check @Or.elim
--  $\forall \{a b c : \text{Prop}\}, a \vee b \rightarrow (a \rightarrow c) \rightarrow (b \rightarrow c) \rightarrow c$ 

-- v-elim, then v-intro-right and v-intro-left
example (A B : Prop) (h : A ∨ B) : B ∨ A :=
  h.elim (fun a => Or.inr a) (fun b => Or.inl b)
example (A B : Prop) (h : A ∨ B) : B ∨ A := -- the same, by pattern
  match h with
  | .inl a => .inr a
  | .inr b => .inl b
```

`Or.elim` is `v-elim-sec` with its three propositions made implicit. `match` is the same rule read as case analysis: one arm per constructor.

Biimplication

I Definition

$A \leftrightarrow B := (A \Rightarrow B) \wedge (B \Rightarrow A).$

Nothing to derive: the rules of $\leftrightarrow$ are the rules of $\wedge$ applied to two implications.

N&G Rem. 7.3.1

```
-- Rem. 7.3.1:  $A \leftrightarrow B := (A \Rightarrow B) \wedge (B \Rightarrow A).$  Lean: a structure with
two fields.
#print Iff                                -- structure Iff (a b : Prop) : Prop
--      fields: Iff.mp : a → b    Iff.mpr : b → a
#check @Iff.intro                          --  $\forall \{a b : Prop\}, (a \rightarrow b) \rightarrow (b \rightarrow a) \rightarrow (a \leftrightarrow b)$ 

example (A B : Prop) (h : A  $\leftrightarrow$  B) (a : A) : B := h.mp a
example (A B : Prop) : (A  $\wedge$  B)  $\leftrightarrow$  (B  $\wedge$  A) :=
  {fun h => ⟨h.2, h.1⟩, fun h => ⟨h.2, h.1⟩}
```

A proof in propositional logic: $(A \vee B) \Rightarrow (\neg A \Rightarrow B)$

(a)	$A : * \quad B : *$	
(c)	$x : \Pi C : *. ((A \rightarrow C) \rightarrow (B \rightarrow C) \rightarrow C)$	
(d)	$y : A \rightarrow \perp$	
(1)	$x B : (A \rightarrow B) \rightarrow (B \rightarrow B) \rightarrow B$	(<i>appl</i>) on (c), (a)
(e)	$u : A$	
(2)	$y u : \perp$	(<i>appl</i>) on (d), (e)
(3)	$y u B : B$	(<i>appl</i>) on (2), (a)
(4)	$\lambda u : A. y u B : A \rightarrow B$	(<i>abst</i>) on (3)
(5)	$x B (\lambda u : A. y u B) : (B \rightarrow B) \rightarrow B$	(<i>appl</i>) on (1), (4)
(f)	$v : B$	
(6)	$v : B$	(<i>var</i>) on (f)
(7)	$\lambda v : B. v : B \rightarrow B$	(<i>abst</i>) on (6)
(8)	$x B (\lambda u : A. y u B) (\lambda v : B. v) : B$	(<i>appl</i>) on (5), (7)
(9)	$\lambda y : A \rightarrow \perp. x B (\lambda u : A. y u B) (\lambda v : B. v) : (A \rightarrow \perp) \rightarrow B$	(<i>abst</i>) on (8)
(10)	$\lambda x : A \vee B. \lambda y : A \rightarrow \perp. \dots : (A \vee B) \rightarrow (A \rightarrow \perp) \rightarrow B$	(<i>abst</i>) on (9)

The one step that needs an idea is (1): apply x to the goal B itself, since the result then ends in B .

N&G §7.3, Fig. 7.1

The same proof, as one term

```
-- the proof object of  $(A \vee B) \Rightarrow (\neg A \Rightarrow B)$ , line (10) of the derivation
theorem or_neg (A B : Prop) : A  $\vee$  B  $\rightarrow$   $\neg$ A  $\rightarrow$  B :=
  fun x y => x.elim (fun u => (y u).elim) (fun v => v)

#print or_neg
-- theorem or_neg :  $\forall$  (A B : Prop), A  $\vee$  B  $\rightarrow$   $\neg$ A  $\rightarrow$  B :=
-- fun A B x y => Or.elim x (fun u => False.elim (y u)) fun v => v
```

line of the derivation	subterm	rule
(2), (3)	(y u).elim	(appl) twice; $\perp$ -elim is .elim
(4)	fun u => (y u).elim	(abst)
(7)	fun v => v	(abst)
(1), (5), (8)	x.elim (...) (...)	(appl) three times: $\vee$ -elim
(9), (10)	fun x y => ...	(abst) twice

The term is the definiens of line (10), and Lean checks it against the proposition.

Classical logic

Everything so far is constructive logic. Classical logic has two more laws, and neither is derivable from the rules given so far:

$$\text{ET: } A \vee \neg A \qquad \text{DN: } \neg\neg A \Rightarrow A.$$

Constructive logic with ET derives DN, and with DN derives ET, so adding one suffices.

How to add ET to λC

Not a new rule: an assumed inhabitant, as a flag in front of every context.

$$i_{ET} : \Pi\alpha : * . \alpha \vee \neg\alpha$$

An axiom is a variable of the proposition's type, declared once and never abstracted away.

Every proof under that flag is classical, and says so: i_{ET} occurs in its term. A proof that does not mention i_{ET} is constructive.

N&G §7.4

ET gives DN: the derivation, from the goal

Goal: $? : \Pi\beta : * . \neg\neg\beta \rightarrow \beta$ under the flag i_{ET} . Two flags open, and $i_{ET} \beta \beta$ leaves two implications to prove.

(a)	$i_{ET} : \Pi\alpha : * . \alpha \vee \neg\alpha$	
(b)	$\beta : *$	
(c)	$x : \neg\neg\beta$	
(1)	$i_{ET} \beta : \beta \vee \neg\beta$	(<i>appl</i>) on (a), (b)
(2)	$i_{ET} \beta : \Pi\gamma : * . (\beta \rightarrow \gamma) \rightarrow (\neg\beta \rightarrow \gamma) \rightarrow \gamma$	the coding of $\vee$, unfolded
(3)	$i_{ET} \beta \beta : (\beta \rightarrow \beta) \rightarrow (\neg\beta \rightarrow \beta) \rightarrow \beta$	(<i>appl</i>) on (2), (b)
(4)	$\lambda y : \beta . y : \beta \rightarrow \beta$	(<i>abst</i>)
(5)	$i_{ET} \beta \beta (\lambda y : \beta . y) : (\neg\beta \rightarrow \beta) \rightarrow \beta$	(<i>appl</i>) on (3), (4)
(d)	$z : \neg\beta$	
(6)	$x z : \perp$	(<i>appl</i>) on (c), (d)
(7)	$x z \beta : \beta$	(<i>appl</i>) on (6), (b)
(8)	$\lambda z : \neg\beta . x z \beta : \neg\beta \rightarrow \beta$	(<i>abst</i>) on (7)
(9)	$i_{ET} \beta \beta (\lambda y : \beta . y) (\lambda z : \neg\beta . x z \beta) : \beta$	(<i>appl</i>) on (5), (8)

Two more (*abst*) close (c) and (b). Line (2) is (*conv*): the abbreviation $\vee$ is unfolded so that (*appl*) can see the Π .

N&G §7.4

The excluded third, in Lean

As a hypothesis: the flag i_{ET} is a parameter. The proof depends on no axiom; it is constructive relative to `em`.

```
-- the excluded third as a flag in front of the context: a parameter em
theorem dn (q : Prop) (em : ∀ p : Prop, p ∨ ¬p) (h : ¬¬q) : q :=
  (em q).elim (fun y => y) (fun z => (h z).elim)
#print axioms dn      -- 'dn' does not depend on any axioms
```

As a library constant: `Classical.em` discharges the flag. Now the proof depends on three axioms, and none is called “`em`”.

```
#check Classical.em      -- Classical.em (p : Prop) : p ∨ ¬p
#check @Classical.byContradiction -- ∀ {p : Prop}, (¬p → False) → p
theorem dn' (q : Prop) (h : ¬¬q) : q := dn q Classical.em h
#print axioms dn'
-- 'dn'' depends on axioms: [propext, Classical.choice, Quot.sound]
```

Lean does not assume the excluded third; it proves it from choice, propositional extensionality and quotients. One flag on the board, three in the kernel: what they are is next lecture.

Existence

$\forall$ is Π itself. For $\exists$ the first-order definition $\neg \forall_{x \in S} (\neg P(x))$ holds only classically; the second-order coding works in constructive logic.

Definition. $\exists_{x \in S} (P(x)) := \Pi \alpha: * . ((\Pi x: S . P x \rightarrow \alpha) \rightarrow \alpha)$.

for all α : if for every x in S , $P(x)$ implies α , then α .

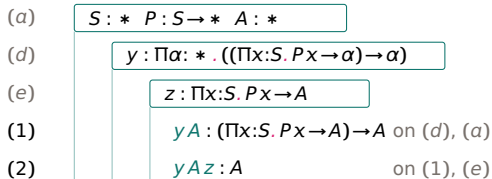
That is the elimination rule, and it says how an existence is used:

$$\frac{a \in S \quad P(a)}{\exists_{x \in S} (P(x))} \text{ (\exists-intro)} \qquad \frac{\exists_{x \in S} (P(x)) \quad \forall_{x \in S} (P(x) \Rightarrow A)}{A} \text{ (\exists-elim)} \quad x \notin \text{FV}(A)$$

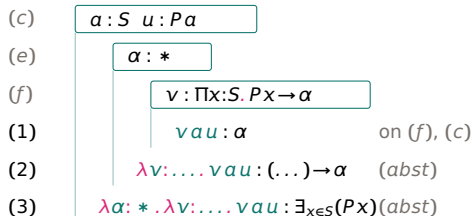
The mathematician's shortcut: “there is an x with $P(x)$; let x be such that $P(x)$ holds; ...; hence A .” The x of the second sentence has a scope, and A must not mention it. The rule makes that scope a flag. N&G §7.5, Rem. 7.5.1

$\exists$ -elim-sec and $\exists$ -intro-sec, derived

Elimination. From y , an existence, and z , a function on witnesses: two applications.



Introduction. From $\alpha : S$ and $u : Pa$, build a function of α and v :



Why $x \notin \text{FV}(A)$ is essential: in line (1) of the elimination, A is substituted for α in a type where x is bound. If A mentioned x , that x would be captured, the defect L1's substitution was built to avoid.

N&G §7.5, Rem. 7.5.2, Ex. 7.11

Exists, in Lean

```
--  $\exists x:S. P x := \prod \alpha:*. ((\prod x:S. P x \rightarrow \alpha) \rightarrow \alpha)$ 
def Ex2 (S : Type) (P : S → Prop) : Prop :=
   $\forall \alpha : \text{Prop}, (\forall x : S, P x \rightarrow \alpha) \rightarrow \alpha$ 
theorem Ex2.intro {S : Type} {P : S → Prop} (a : S) (u : P a) : Ex2 S P :=
  fun _ v => v a u
theorem Ex2.elim {S : Type} {P : S → Prop} {A : Prop}
  (y : Ex2 S P) (z :  $\forall x, P x \rightarrow A$ ) : A := y A z

#print Exists          -- inductive Exists : ( $\alpha \rightarrow \text{Prop}$ ) → Prop
                        --   Exists.intro (w :  $\alpha$ ) : p w → Exists p
example :  $\exists n : \text{Nat}, n + n = 6$  := {3, rfl}      -- the witness, then the proof
example (S : Type) (P : S → Prop) (h :  $\exists x, P x$ ) (k :  $\forall x, P x \rightarrow \text{False}$ ) :
  False :=
  match h with
  | (w, hw) => k w hw      --  $\exists$ -el: open the flag
```

$\langle 3, \text{rfl} \rangle$ is $\exists$ -intro-sec with the witness first; match is $\exists$ -elim-sec, and its arm is the flag $x : S, u : P x$ of $\exists$ -elim.

The witness stays inside the flag

```
-- the conclusion A may not mention the witness x.
-- Lean's version of the side condition is a sort: an Exists in Prop may
-- only be eliminated into a Prop, so no function can return the witness.
#guard_msgs(drop error) in
def witness (h :  $\exists n : \text{Nat}, n + n = 6$ ) : Nat :=
  match h with
  |  $\langle w, - \rangle$  => w
-- error: recursor 'Exists.casesOn' can only eliminate into Prop

theorem witness' (h :  $\exists n : \text{Nat}, n + n = 6$ ) :  $\exists m : \text{Nat}, m + m + 2 = 8$  :=
  match h with
  |  $\langle w, hw \rangle$  =>  $\langle w, \text{by omega} \rangle$ 
-- into Prop: accepted
```

The side condition $x \notin \text{FV}(A)$ keeps the witness out of the conclusion. Lean enforces it by sort: an Exists in Prop is eliminated into a Prop only. The full reason comes with inductive types.

A proof in predicate logic: $\neg\exists_{x\in S} (P(x)) \Rightarrow \forall_{y\in S} (\neg P(y))$

Three flags for the three leading binders; then a $\perp$ is wanted, and the only way to one is u , applied to an existence built from y and v .

(a)	$S : * \quad P : S \rightarrow *$	
(c)	$u : \neg(\exists x:S. Px)$	
(d)	$y : S$	
(e)	$v : Py$	
(1)	$\lambda\alpha : * . \lambda w : (\Pi x:S. Px \rightarrow \alpha). wyv : \exists x:S. Px$	$\exists$ -intro-sec
(2)	$u(\lambda\alpha : * . \lambda w : \dots wyv) : \perp$	(<i>appl</i>) on (c), (1)
(3)	$\lambda v:Py. u(\dots) : \neg(Py)$	(<i>abst</i>) on (2)
(4)	$\lambda y:S. \lambda v:Py. u(\dots) : \forall y:S. \neg(Py)$	(<i>abst</i>) on (3)
(5)	$\lambda u:\neg(\exists \dots). \lambda y:S. \lambda v:Py. u(\dots) : \neg(\exists x:S. Px) \Rightarrow \forall y:S. \neg Py$	(<i>abst</i>) on (4)

$\forall$ and $\exists$ are written for Π and the coding from here on. Line (1) is a hidden $\exists$ -introduction: nothing in the term says so.

N&G §7.6

The same proof, as one term

```
-- the proof object of  $\neg \exists x:S. P x \Rightarrow \forall y:S. \neg P y$ , line (5) of the derivation
theorem not_ex (S : Type) (P : S → Prop) :  $\neg(\exists x, P x) \rightarrow \forall y, \neg P y$  :=
  fun u y v => u ⟨y, v⟩

#print not_ex
-- theorem not_ex :  $\forall (S : Type) (P : S \rightarrow Prop), (\neg \exists x, P x) \rightarrow \forall (y : S), \neg P y$  :=
-- fun S P u y v => u (Exists.intro y v)

-- with the coding instead of Lean's Exists, line (1) reappears:
theorem not_ex' (S : Type) (P : S → Prop) :  $\neg(\text{Ex}_2 S P) \rightarrow \forall y, \neg P y$  :=
  fun u y v => u (fun _ w => w y v)
```

line of the derivation	in Lean	
(1) $\lambda\alpha:*. \lambda w:.... wyv$	$\langle y, v \rangle$	the anonymous constructor is $\exists$ -intro-sec
(2) $u(...)$	$u \langle y, v \rangle$	(<i>appl</i>)
(3)–(5)	$\text{fun } u \ y \ v \Rightarrow$	(<i>abst</i>) three times

Terms grow, and the rules hide

Two things went wrong in the last derivation, and both get worse with size.

the terms grow

$\neg\exists x:S. P\ x \Rightarrow \forall y:S. \neg P\ y$ one line of mathematics

$\lambda u: (\Pi\alpha:*. ((\Pi x:S. P\ x \rightarrow \alpha) \rightarrow \alpha) \rightarrow \perp).$
 $\lambda y:S. \lambda v:P\ y.$ the coding of $\exists$, copied twice
 $u\ (\lambda\alpha:*. \lambda w: (\Pi x:S. P\ x \rightarrow \alpha). w\ y\ v)$

three lines of λC

the rules hide

$\lambda\alpha:*. \lambda w: (\Pi x:S. P\ x \rightarrow \alpha).$
 $w\ y\ v : \exists x:S. P\ x$

this is $\exists$ -intro

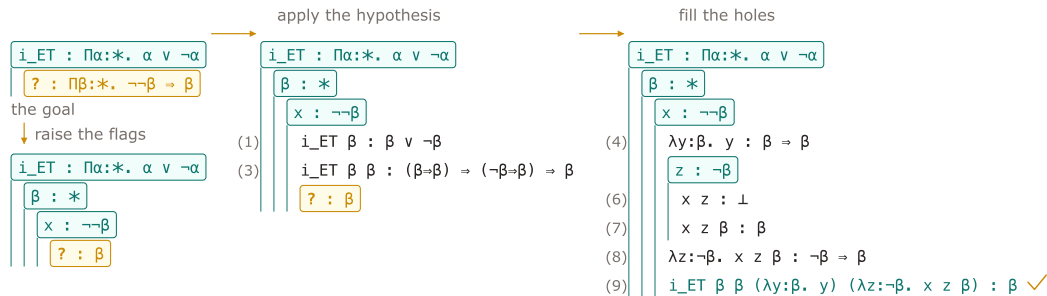
and nothing in the term says so:
two λ s and an application

Two remedies. Names: write $\exists\text{-in}(S, P, y, v)$ for line (1); next lecture. A machine for the bookkeeping: goal, flags, holes; the rest of this one.

N&G §7.6, §7.7

Derivations are built from the goal

Written top to bottom, found bottom to top: a goal $A \rightarrow B$ or $\Pi x:A. B$ raises a flag; a hypothesis applied backward turns the goal into its premisses.



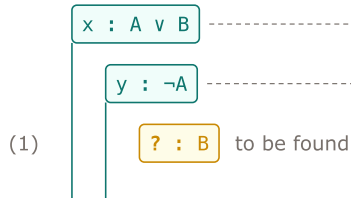
Term Finding, by a person with a strategy; no algorithm does it.

N&G §2.6, §7.3, §7.4

Tactic mode is that bookkeeping, mechanised

After `by`, Lean holds the ? line and shows it as a goal state: flags above the line, the hole below it.

flag derivation, unfinished



Lean's goal state




the same object

flags are the hypotheses
`intro x` (*abst*), read bottom to top
`apply h` (*appl*), read bottom to top
`exact t` the hole is filled outright

the ? line is the goal
the goal $\prod x:A. B$ becomes B , with a flag $x : A$
with $h : A \rightarrow B$, the goal B becomes A
 t must have the type of the goal N&G Rem. 2.4.7

intro raises a flag, exact fills the hole

```
theorem diag' (S : Type) (Q : S → S → Prop) :  
  (∀ x y : S, Q x y) → ∀ u : S, Q u u := by  
  intro z      -- z : ∀ x y, Q x y          ⊢ ∀ u, Q u u  
  intro u      -- z : ∀ x y, Q x y, u : S    ⊢ Q u u  
  exact z u u  -- no goals
```



```
#print diag'  
-- theorem diag' : ∀ (S : Type) (Q : S → S → Prop),  
--   (∀ (x y : S), Q x y) → ∀ (u : S), Q u u :=  
--   fun S Q z u => z u u
```

The comments are the goal state after each line. What by produced is the term L3 wrote by hand.

apply leaves a hole

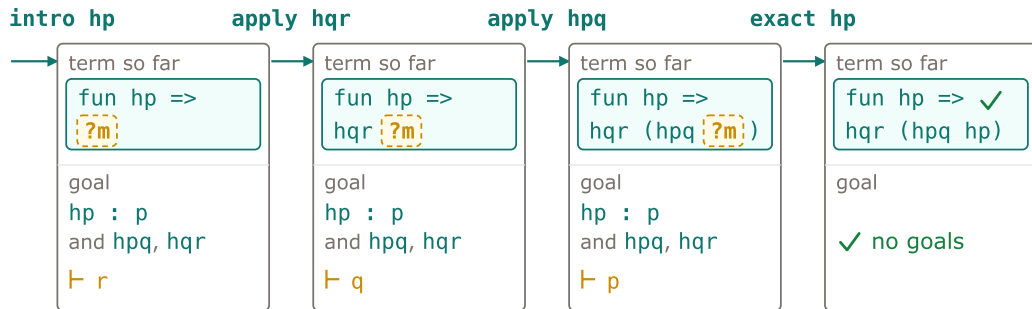
A hole has a name, ?m, and `apply h` fills the hole with `h ?m`. The new hole is the next goal.

```
theorem chain (p q r : Prop) (hpq : p → q) (hqr : q → r) :  
  p → r := by  
  intro hp      -- ⊢ r      term so far:  fun hp => ?m  
  apply hqr      -- ⊢ q      fun hp => hqr ?m  
  apply hpq      -- ⊢ p      fun hp => hqr (hpq ?m)  
  exact hp       -- no goals  fun hp => hqr (hpq hp)
```

```
#print chain  
-- theorem chain : ∀ (p q r : Prop), (p → q) → (q → r) → p → r :=  
--   fun p q r hpq hqr hp => hqr (hpq hp)
```

Each `apply` is one (*appl*) read upward: the conclusion is known, a premiss is left to find.

The term grows as the goals close



`apply h`: the hole becomes `h ?m`, and the new hole is the new goal

One tactic per rule

Each connective has a rule to prove it and a rule to use it, and each rule is one tactic. The table is this lecture, read as a script.

	to prove it (introduction)	to use h (elimination)
$A \Rightarrow B$	intro a	apply h, exact h a
$\forall x:S. Px$	intro x	apply h, exact h a
$A \wedge B$	constructor, exact $\langle a, b \rangle$	obtain $\langle a, b \rangle := h, h.1, h.2$
$A \vee B$	left, right	rcases h with a b
$\neg A$	intro a	exact h a, exact absurd a h
$\perp$	—	exact h.elim, contradiction
$\exists x:S. Px$	exact $\langle w, p \rangle$, exists w	obtain $\langle w, hw \rangle := h$
$A \Leftrightarrow B$	constructor	h.mp, h.mpr

obtain and rcases raise the flags of an elimination rule: for $\exists$, the $x : S$ and $u : Px$ of the book's strategy; for $\vee$, one goal per case.

TPIL §5.1–5.3

$\wedge$, $\vee$, $\exists$: the rules as tactics

```
example (A B : Prop) (h : A  $\wedge$  B) : B  $\wedge$  A := by
  obtain ⟨a, b⟩ := h      --  $\wedge$ -el: two flags a : A, b : B
  constructor            --  $\wedge$ -in: two goals, B and A
  · exact b
  · exact a

example (A B : Prop) (h : A  $\vee$  B) : B  $\vee$  A := by
  rcases h with a | b    --  $\vee$ -el: one goal per case
  · right; exact a        --  $\vee$ -in2
  · left;  exact b        --  $\vee$ -in1

example (S : Type) (P Q : S → Prop) (h :  $\exists$  x, P x  $\wedge$  Q x) :  $\exists$  x, Q x := by
  obtain ⟨w, _, hq⟩ := h  --  $\exists$ -el then  $\wedge$ -el: flags w : S, hq : Q w
  exact ⟨w, hq⟩          --  $\exists$ -in
```

A bullet `·` focuses one goal: the flag indentation of a derivation, written with a dot.

$\neg$, $\perp$, $\forall$, $\Leftrightarrow$: the rules as tactics

```
example (A : Prop) (h :  $\neg$ A) (a : A) : False := by exact h a      --  $\neg$ -el
example (A : Prop) (a : A) :  $\neg\neg$ A := by                          --  $\neg$ -in
  intro h; exact h a
example (A : Prop) (h : False) : A := by exact h.elim            --  $\perp$ -el
example (A : Prop) (h : False) : A := by contradiction          -- the same

example (S : Type) (P : S  $\rightarrow$  Prop) (h :  $\forall$  x, P x) (a : S) : P a := by
  apply h                                                         --  $\forall$ -el
example (S : Type) (P : S  $\rightarrow$  Prop) (h :  $\forall$  x, P x  $\wedge$  P x) :  $\forall$  y, P y := by
  intro y; exact (h y).1                                           --  $\forall$ -in

example (A B : Prop) (f : A  $\rightarrow$  B) (g : B  $\rightarrow$  A) : A  $\leftrightarrow$  B := by
  constructor                                                       --  $\leftrightarrow$ -in
  · exact f
  · exact g
```

The propositional proof, as a script

```
theorem or_neg' (A B : Prop) :  
  A ∨ B → ¬A → B := by  
  intro x y      -- flags (c), (d)  
  apply x.elim    -- v-el, backwards  
  · intro u       -- flag (e)  
    exact absurd u y -- (2), (3)  
  · intro v       -- flag (f)  
    exact v        -- (6)  
  
#print or_neg'  
-- theorem or_neg' : ∀ (A B : Prop),  
--   A ∨ B → ¬A → B :=  
-- fun A B x y =>  
--   Or.elim x (fun u => absurd u y)  
--   fun v => v
```

Each tactic is one line of the derivation, read upward: `intro` raises the flags, `apply` `x.elim` is $\vee$ -elimination with its two premisses left as goals, and each `exact` fills a hole with a term of the derivation.

The term `#print` shows is the one of the slide “The same proof, as one term”, with `absurd` for the two applications (2), (3).

`cases x with | inl u => ... | inr v => ...` spells the same $\vee$ -elimination as case analysis; its term is the recursor of `Or`, `L7`'s `business`.

Three views of one proof

the script

```
intro x y
```

```
cases x with
```

```
| inl u => exact absurd u y
```

```
| inr v => exact v
```

the term it builds

```
fun x y => x.elim (fun u => absurd u y) (fun v => v)
```

intro cases inl branch inr branch

the goal after the line

```
x : A v B  
y : ¬A  
├ B
```

```
u : A  
y : ¬A  
├ B
```

```
u : A  
y : ¬A  
├ B
```

```
v : B  
y : ¬A  
├ B
```

✓ closed

```
v : B  
y : ¬A  
├ B
```

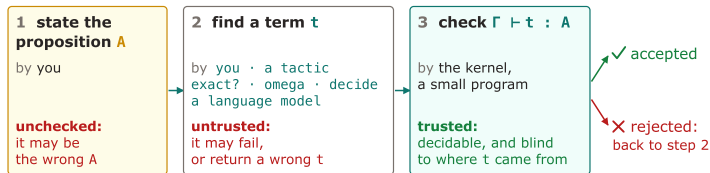
✓ closed

#print shows this term; the kernel checks only this

The script is the derivation read from the bottom; the goal after each line is the flag derivation with a ? line; the term is what the kernel checks. One object, three views.

AI checkpoint: search is untrusted, checking is not

A tactic, a library search, a decision procedure and a language model do one job: propose a term. The kernel checks it, whoever proposed it.



Term Finding is undecidable, so step 2 has no algorithm;
Type Checking is decidable, so step 3 needs no trust in step 2.

What the check cannot catch

A wrong statement. A checked proof of the wrong proposition is still a checked proof.

What is still missing

missing	consequence	added by
names	the coding of $\exists$ is copied into every proof that uses it; line (1) of §7.6 hides an $\exists$ -introduction	definitions (L5)
equality, order, sets	everything so far is pure logic; $x = y$ and $x \leq y$ are not in the system	equality (L6)
numbers, with induction	$\mathbb{N}$, 0, s can be assumed, but no coding gives the induction principle	inductive types (L7)

Next lecture

Definitions: a name is a step of the type theory, the environment is the book, and a proof step is a definition too. N&G §7.7, §8.1

References I

Avigad, J., L. de Moura, S. Kong, and S. Ullrich (2026). *Theorem Proving in Lean 4*. With contributions from the Lean community; tested against Lean 4.33.0. URL: https://lean-lang.org/theorem_proving_in_lean4/ (visited on 08/20/2026).

Coquand, T. and G. Huet (1988). “The calculus of constructions”. In: *Information and Computation* 76.2–3, pp. 95–120.

Gentzen, G. (1935). “Untersuchungen über das logische Schließen”. In: *Mathematische Zeitschrift* 39, pp. 176–210, 405–431.

Howard, W. A. (1980). “The formulae-as-types notion of construction”. In: *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*. Ed. by J. P. Seldin and J. R. Hindley. Academic Press, pp. 479–490.

Nederpelt, R. and H. Geuvers (2014a). “The encoding of logical notions in λC ”. In: *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press. Chap. 7, pp. 137–164.

References II

Nederpelt, R. and H. Geuvers (2014b). *Type Theory and Formal Proof: An Introduction*. Cambridge: Cambridge University Press.

Chapter 7 of the book throughout: §7.1 absurdity and negation (Rem. 7.1.1, 7.1.2), §7.2 conjunction and disjunction (the derivations of pp. 141 and 143, Rem. 7.2.1), §7.3 the propositional example (Fig. 7.1, Rem. 7.3.1), §7.4 classical logic, §7.5 the existential quantifier (Rem. 7.5.1–7.5.3), §7.6 the predicate-logic example, §7.7 the conclusions. Exercises 7.4, 7.7 and 7.11 are answered on the slides. The Lean slides follow *Theorem Proving in Lean 4*: ch. 3 (propositions and proofs), §4.4 (the existential quantifier), §5.1–5.3 (entering tactic mode, the basic tactics), and §7.3 for And, Or, Exists as inductive types.